
PROJET DE FIN DE MODULE

Machine Learning

Phase 1

Cadrage du projet et constitution du dataset

1. Objectifs de la phase

À l'issue de cette phase, vous devez avoir réalisé les éléments suivants :

1. Choisir un **sujet** de projet **pertinent** et un **domaine métier** clairement identifié.
2. Sélectionner une ou plusieurs **APIs publiques et gratuites** comme **source de données**.
3. Définir vos **objectifs métiers** et les avoir traduits en **objectifs ML mesurables**.
4. Construire un **dataset déséquilibré** respectant les contraintes imposées.
5. **Documenter le dataset** de manière professionnelle.

2. Contraintes à respecter impérativement

2.1. Nature du problème

Votre projet doit porter sur un **problème de classification supervisée** (binaire ou multi-classes). Les problèmes de **régression** et de **clustering** ne sont **pas acceptés** pour ce projet.

2.2. Source des données

La constitution du dataset doit être réalisée via interrogation d'une ou plusieurs **APIs publiques et gratuites**.

2.3. Caractéristiques obligatoires du dataset

Critère	Exigence
Type de tâche	Classification supervisée
Taille totale	≥ 10 000 lignes
Nombre de features	≥ 8 après feature engineering
Classe minoritaire (ratio)	Entre 5 % et 25 % du total
Types de variables	Mélange de numériques et catégorielles

2.4. APIs recommandées

Vous pouvez utiliser l'une des APIs suivantes ou toute autre API publique et gratuite :

- **Culture** : TMDb (films), Spotify Web API (musique), OpenLibrary (livres), Jikan (anime/manga).
- **Sport** : Football-Data.org, TheSportsDB, OpenLigaDB.
- **Finance** : CoinGecko (crypto), Alpha Vantage (bourse), Exchange Rates API, World Bank API.
- **Environnement** : OpenWeather, OpenAQ (qualité de l'air), NASA APIs, USGS (séismes).

- **Transport** : OpenSky Network (vols), SNCF Open Data, RATP.
- **Santé** : Open Food Facts, disease.sh, PubMed API.
- **Web et social** : Reddit API (PRAW), GitHub API, Hacker News API, Wikipedia API.

3. Travail à réaliser

3.1. Étape 1 — Exploration et choix du sujet

Activités

- Identifier un **domaine métier** qui vous intéresse.
- Explorer **2 à 3 APIs candidates** : lire leur documentation, tester quelques endpoints via Postman, curl, ou un notebook Python, vérifier les quotas gratuits et les licences.
- Formuler **2 à 3 questions métier** possibles pour chaque API.
- Choisir la combinaison API + question métier la plus prometteuse.

Pièges à éviter

- Choisir une API dont la version gratuite limite à 100 requêtes/jour si vous avez besoin de 10 000 lignes.
- Choisir un sujet où la variable cible n'existe pas naturellement dans les données (vous devriez l'inventer).
- Choisir un sujet naturellement équilibré puis forcer artificiellement un déséquilibre.

3.2. Étape 2 — Définition des objectifs métiers et ML

C'est le travail intellectuel le plus important de la phase.

a) Objectifs métiers

Formuler clairement **les objectifs métiers** quantifiés de votre projet.

b) Traduction en objectifs ML

C'est l'exercice clé du cadrage. Vous devez produire un **tableau de correspondance** entre vos objectifs métiers et vos objectifs ML :

Objectif métier	Objectif ML	Métrique principale
Détecter 80 % des échecs ...	Maximiser le rappel sur la classe « échec »	Recall $\geq 0,80$
Limiter les ...	Maintenir une précision correcte	Precision $\geq 0,50$

c) Analyse du coût métier asymétrique

Question fondamentale : **que coûte plus cher, un faux positif ou un faux négatif ?**

- **Exemple fraude** : un **faux négatif** (fraude non détectée) coûte des milliers d'euros ; un **faux positif** (transaction légitime bloquée) coûte quelques euros de friction client → on privilégie le **recall**.

Vous devez **chiffrer** (même approximativement) cette asymétrie, car elle orientera le choix de la **métrique principale** et le choix du **seuil de décision**.

d) Métriques à utiliser

Votre **métrique principale** doit être adaptée au déséquilibre.

- ✓ **Acceptées** : PR-AUC, F1-score, recall, precision, ...
- ✗ **Refusées comme métrique principale** : **accuracy seule**, ROC-AUC seule (elle est optimiste sur les données déséquilibrées).

3.3. Étape 3 — Construction du dataset

Activités

Développez un script Python `data_collection.py` **reproductible** qui :

- Gère l'**authentification** de l'API.
- Gère la **pagination** pour récupérer tous les résultats nécessaires.
- Respecte le **rate limiting** de l'API.
- Gère les **erreurs réseau** et les codes HTTP.
- **Sauvegarde les données brutes** au format **JSON** ou **Parquet** pour éviter de re-requêter à chaque exécution.
- **Transforme les données brutes** en **DataFrame** tabulaire exploitable.
- Génère un **fichier CSV** ou **Parquet final** exploitable en Phase 2.

Conseils pratiques

- **Sauvegardez régulièrement** votre dataset partiel en cas d'interruption.
- **Loguez chaque appel API** (timestamp, endpoint, statut) pour faciliter le debug.
- **Testez d'abord sur un petit échantillon (100 lignes)** avant de lancer la collecte complète.

3.4. Étape 4 — Documentation du dataset

Rédigez un fichier **DATASET.md** contenant les sections suivantes :

a) Identification

- **Nom** du dataset, auteur(s), date de collecte, version.

b) Source

- API(s) utilisée(s) avec URL et endpoints interrogés.
- Date d'accès.

c) Description

- Objectif du dataset (problème métier qu'il doit résoudre).
- Nombre de lignes, nombre de colonnes.
- Schéma détaillé : pour chaque variable, son nom, type, description métier, plage de valeurs, unité.
- Identification claire de la variable cible.
- Distribution des classes (avec graphique).

4. Livrables de fin de phase

Déposez sur votre **dépôt Git** les éléments suivants :

N°	Livrable	Contenu attendu
1	Fiche de cadrage (cadrage.md ou .pdf)	Objectifs métiers quantifiés, tableau de traduction métier → ML, analyse du coût asymétrique, choix des métriques justifié.
2	Script de collecte (src/data_collection.py)	Reproductible et documenté (docstrings, commentaires).
3	Dataset constitué (data/dataset.csv ou .parquet)	Respectant toutes les contraintes de la section 2.3. Accompagné d'un extrait (data/sample.csv avec 100 lignes) pour vérification rapide.
4	Documentation du dataset (DATASET.md)	Complète selon le plan de la section 3.4
5	Notebook exploratoire initial (notebooks/01_discovery.ipynb)	Chargement du dataset, df.info(), df.describe(), df.head(), vérification du déséquilibre (value_counts + graphique) confirmant que le dataset est exploitable.

5. Modalités de validation

Vous présenterez, en **10 minutes**, les éléments suivants :

- Votre **sujet** et votre **question métier**.
- Votre **API source** et votre **script de collecte** fonctionnel.
- La **distribution** de votre **variable cible** (preuve du déséquilibre).
- Votre tableau de **traduction métier → ML**.

6. Conseils pour réussir cette phase

Bonnes pratiques

- **Ne sous-estimez pas le temps** de la collecte de données : c'est souvent 2× plus long que prévu.
- **Commencez petit** : collectez 100 lignes avant d'en collecter 10 000.
- **Vérifiez le déséquilibre tôt** : si votre variable cible n'est pas déséquilibrée, ajustez avant d'aller plus loin.
- **Documentez au fur et à mesure**, pas à la fin.
- **Commitez régulièrement sur Git** : un commit par sous-étape au minimum.